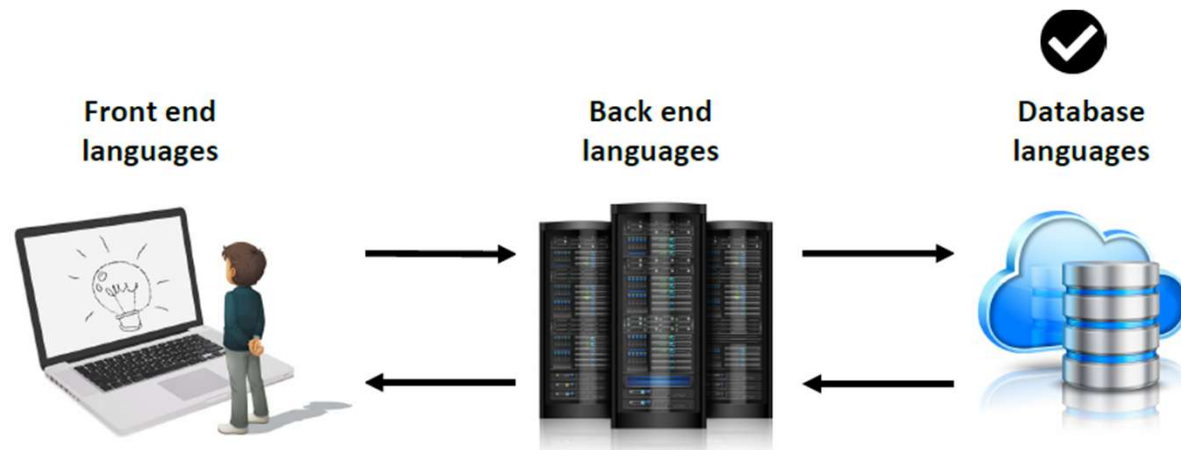




mongoDB®

MongoDB

Introduction



Introduction

- ❖ A database Management System provides the mechanism to store and retrieve the data.
- ❖ There are different kinds of database Management Systems:
 1. RDBMS (Relational Database Management Systems)
 2. OLAP (Online Analytical Processing)
 3. NoSQL (Not only SQL)

Limitations of RDBMS

- ❖ In relational database we need to define structure and schema of data first and then only we can process the data.
- ❖ Relational database systems provides consistency and integrity of data by enforcing ACID properties (Atomicity, Consistency, Isolation and Durability). However in most of the other cases these properties are significant performance overhead and can make your database response very slow.
- ❖ Most of the applications store their data in JSON format and RDBMS don't provide you a better way of performing operations such as create, insert, update, delete etc on this data.

Introduction to NoSQL

10gen | the
MongoDB
company

What is NoSQL Database?

- ❖ Not Only SQL (NoSQL) or non relational databases provides a mechanism for storage and retrieval of data other than tabular relations model used in relational databases.
- ❖ NoSQL database doesn't use tables for storing data.
- ❖ It is generally used to store big data and real-time web applications.

Why NoSQL Database?

- ❖ NoSQL databases were created in response to the limitations of traditional relational database technology.
- ❖ **Flexible Schema:** Allow for dynamic schemas, accommodating unstructured or semi-structured data without predefined structures.
- ❖ **Duplicacy over Relations:** Duplicates data across records (denormalization) to enhance performance and scalability, rather than relying on complex relationships and joins as in relational databases.
- ❖ **Horizontal Scalability:** Designed to scale out by adding more servers, handling large volumes of data efficiently.
- ❖ **Performance:** Optimized for high throughput and low latency, suitable for real-time applications.

When to use NoSQL Database?

- ❖ When you want to store and retrieve huge amount of data.
- ❖ The relationship between the data you store is not that important.
- ❖ The data is not structured and changing over time.
- ❖ Constraints and Joins support is not required at database level.
- ❖ The data is growing continuously and you need to scale the database regular to handle the data.

Where to use NoSQL Database?

- ❖ Data isn't relational (e.g. Documents)
- ❖ Too much data to fit in a relational database

SQL vs NoSQL

SQL Database	NoSQL Database
SQL databases are primarily called RDBMS or Relational Databases	NoSQL databases are primarily called as Non-relational or distributed database
SQL databases are table based databases	NoSQL databases can be document based, key-value pairs, graph databases
MySQL uses SQL to query database	MongoDB uses BSON to query database
Developed in the 1970s with a focus on reducing data duplication	Developed in the late 2000s with a focus on scaling and allowing for rapid application change driven by agile and DevOps practices.
Oracle, MySQL, Microsoft SQL Server, and PostgreSQL	Document: MongoDB and CouchDB, Key-value: Redis and DynamoDB, Wide-column: Cassandra and HBase, Graph: Neo4j and Amazon Neptune

NoSQL

FEATURE DESCRIPTION	COLUMN TITLE	COLUMN TITLE	COLUMN TITLE	COLUMN TITLE
First Row	●	●	●	●
Second Row	●	●	●	●
Third Row	●	●	●	●
Rowmance	●	●	●	●
Rowow	●	●	●	●
Rowanow	●	●	●	●

```
{
  "_id" : ObjectId("5d16f6a1e198c897a4105d0d"),
  "title" : "Networking",
  "description" : "Networking Essentials",
  "author" : "Gaurav Mandes"
}
{
  "_id" : ObjectId("5d16f6a1e198c897a4105d0e"),
  "title" : "Game Engineering",
  "description" : "Game Engineering and Development",
  "author" : "Gaurav Mandes"
}
{
  "_id" : ObjectId("5d16f6a1e198c897a4105d0f"),
  "title" : "PHP",
  "description" : "PHP as a General-Purpose",
  "author" : "Gautam"
}
>
```

Installation (Local)

❖ Official Website for **MongoDB** :

➤ MongoDB Community Server

<https://www.mongodb.com/try/download/community>

➤ MongoDB Compass

<https://www.mongodb.com/try/download/compass>

➤ MongoDB Shell

<https://www.mongodb.com/try/download/shell>

No-Installation (Online)

❖ Official Website for **MongoDB** :

➤ MongoDB Atlas

<https://www.mongodb.com/atlas>

MongoDB

- ❖ Source –available
- ❖ Cross-platform
- ❖ Document-oriented database
- ❖ Editions (Community, Enterprise, Atlas/Cloud)

Organizing Data

➤ Database

➤ Collections

➤ Records

Document store

RDBMS		MongoDB
Database	➡	Database
Table, View	➡	Collection
Row	➡	Document (JSON, BSON)
Column	➡	Field
Index	➡	Index
Join	➡	Embedded Document
Foreign Key	➡	Reference
Partition	➡	Shard

Document store

RDBMS		MongoDB
Database	➡	Database
Table, View	➡	Collection
Row	➡	Document (JSON, B
Column	➡	Field
Index	➡	Index
Join	➡	Embedded Document
Foreign Key	➡	Reference
Partition	➡	Shard

```
> db.user.findOne({age:39})
{
  "_id" : ObjectId("5114e0bd42..."),
  "first" : "Varun",
  "last" : "Dev",
  "age" : 39,
  "interests" : [
    "Reading",
    "Biking "
  ],
  "favorites": {
    "color": "Blue",
    "sport": "Cricket"
  }
}
```

Working with MongoDB

Create Database

- ❖ On MongoDB shell, command for create a database

use database_name;

- ❖ If the database doesn't exist, above command creates a new database otherwise opens the existing one.

Eg: use College;

```
test> use College
switched to db College
College>
```

Delete Database

- ❖ Use command to delete a database:

db.dropDatabase()

```
College> db.dropDatabase()  
{ ok: 1, dropped: 'College' }  
College>
```

List all Databases

- ❖ To list down all the databases, use the command below

`show dbs`

- ❖ This command lists down all the databases and their size on the disk.

```
test> show dbs
admin          40.00 KiB
car_dealership 72.00 KiB
config         84.00 KiB
local         192.00 KiB
```

- ❖ **Note:** *As you can see that the database “College” that we have created is not present in the list of all the databases. This is because a database is not created until you save a document in it*

Collection

- ❖ MongoDB stores documents in collections.
- ❖ Collections are analogous to tables in relational databases.
- ❖ A collection exists within a single database.
- ❖ Collections do not enforce a schema.
- ❖ A collection may store documents those which are not same in structure because its schema free database.

Create Collection –Method 1

- ❖ In MongoDB you need not to create collection before you insert document in it. With a single command you can insert a document in the collection and the MongoDB creates that collection on the fly.

Syntax: `db.collection_name.insert({key:value, key:value...})`

- ❖ Eg: To create a collection in the database College, we use the following command.

`db.student.insert({name:"Amit", age:23, marks:343});`

```
College> db.student.insert({name:'Amit', age:23, marks:343})
DeprecationWarning: Collection.insert() is deprecated. Use insertOne, insertMany, or bulkWrite.
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6989a48bea064713df63b112') }
}
```

Create Collection –Method 2

- ❖ We can also create collection before we actually insert data in it. This method provides you the options that you can set while creating a collection.

Syntax: `db.createCollection(name, options)`

- name is the collection name
- options is an optional field that we can use to specify certain parameters such as size, max number of documents etc. in the collection.

Eg: `db.createCollection("Course");`

```
College> db.createCollection('Course')
{ ok: 1 }
```

```
College> db
College
College> show collections
Course
student
College>
```

Create Collection –Method 2

- ❖ We can also create collection before we actually insert data in it. This method provides you the options that you can set while creating a collection.

Syntax: *db.createCollection(name, options)*

- name is the collection name
- options is an optional field that we can use to specify certain parameters such as size, max number of documents etc. in the collection.

Eg: *db.createCollection("Course", {capped: true,
autoIndexID: true, size: 6142800, max: 10000});*

Deleting the Collection

- ❖ To drop a collection , first connect to the database in which you want to delete collection and then type the following command to delete the collection:

`db.collection_name.drop()`

Eg: db.Course.drop();

```
College> db.Course.drop()  
true  
College> show collections  
student  
College>
```

Documents

- ❖ MongoDB stores data records as BSON documents. BSON is a binary representation of JSON documents.
- ❖ MongoDB documents are composed of field-and-value pairs and have the following structure:

```
{  
  field1: value1,  
  field2: value2,  
  field3: value3,  
  ...  
  fieldN: valueN  
}
```

```
{  
  name: "sue",  
  age: 26,  
  status: "A",  
  groups: [ "news", "sports" ]  
}
```

← field: value
← field: value
← field: value
← field: value

Collection and Documents

```
test> use College
switched to db College
College> db.student.findOne()
{
  _id: ObjectId('6989a48bea064713df63b112'),
  name: 'Amit',
  age: 23,
  marks: 343
}
College>
```

Insert Document

- ❖ There are 3 methods to insert documents into a MongoDB database.

- ✓ `insert()` **// deprecated**
- ✓ `insertOne()`
- ✓ `insertMany()`

insertOne()

- ❖ To insert a single document, use the **insertOne()** method.
- ❖ This method inserts a single object into the database.

Syntax: `db.collection_name.insertOne({key:value, key:value...})`

Insert Document

insertOne()

- ❖ Eg: To insert a document in the collection student (database - College), we use the following command.

```
db.student.insertOne({name:"Varun", age:21, marks:273});
```

```
College> db.student.insertOne({name:"Varun", age:21, marks:273});
{
  acknowledged: true,
  insertedId: ObjectId('698c45d0c770fdf01163b112')
}
```

Insert Document

insertMany()

- ❖ To insert multiple documents at once, use the **insertMany()** method.
- ❖ This method inserts an array of objects into the database.

Syntax: `db.collection_name.insertMany([
 {key:value, key:value...},
 {key:value, key:value...},
 {key:value, key:value...},
 ...])`

Insert Document

insertMany()

- ❖ Eg: To insert multiple documents in the collection student (database – College), we use the following command.

```
db.student.insertMany([{name: "Amar", age: 21, marks: 313},  
                       {name: "Raman", age: 22, marks: 231},  
                       {name: "Suman", age: 24, marks: 273},]);
```

```
College> db.student.insertMany([{name: "Amar", age: 21, marks: 313},  
... {name: "Raman", age: 22, marks: 231},  
... {name: "Suman", age: 24, marks: 273},]);  
{  
  acknowledged: true,  
  insertedIds: {  
    '0': ObjectId('698c46eec770fdf01163b113'),  
    '1': ObjectId('698c46eec770fdf01163b114'),  
    '2': ObjectId('698c46eec770fdf01163b115')  
  }  
}
```

Query Document

- ❖ There are 2 methods to Query/find and select data from a MongoDB collection,
 - ✓ find()
 - ✓ findOne()

find()

- ❖ To select data from a collection in MongoDB, we can use the **find()** method.
- ❖ This method accepts a query object. If left empty, all documents will be returned.

Query Document

find()

Syntax: *db.collection_name.find()*

Ex:

db.student.find()

```
College> db.student.find()
[
  {
    _id: ObjectId('6989a48bea064713df63b112'),
    name: 'Amit',
    age: 23,
    marks: 343
  },
  {
    _id: ObjectId('698b683093d45b3de763b112'),
    name: 'ravi',
    age: 22,
    marks: 222
  },
  {
    _id: ObjectId('698c45d0c770fdf01163b112'),
    name: 'Varun',
    age: 21,
    marks: 273
  },
  {
    _id: ObjectId('698c46eec770fdf01163b113'),
    name: 'Amar',
    age: 21,
    marks: 313
  },
]
```

Query Document

- ❖ The **pretty()** method
- ❖ To display the results in formatted way, **pretty()** method can be used

Syntax: `db.collection_name.find().pretty()`

Query Document

- ❖ The **pretty()** method // **deprecated** in the modern MongoDB Shell (mongosh)

Ex: `db.student.find().pretty()`

```
College> db.student.find().pretty()  
[  
  {  
    _id: ObjectId('698d9929498b373bed63b112'),  
    name: 'Amit',  
    age: 23,  
    marks: 343  
  }  
]
```

Query Document

findOne()

- ❖ To select only one document, we can use the **findOne()** method.
- ❖ This method accepts a query object. If left empty, it will return the first document it finds.

Syntax: `db.collection_name.findOne()`

- ❖ **Note:** This method only returns the first match it finds.

Query Document

findOne()

Ex:

db.student.findOne()

```
College> db.student.findOne()  
{  
  _id: ObjectId('698d9929498b373bed63b112'),  
  name: 'Amit',  
  age: 23,  
  marks: 343  
}
```

Querying Data: where (RDBMS)

- ❖ To query, or filter, data we can include a query in our find() or findOne() methods.

Syntax: `db.collection_name.findOne({<key>: <value>})`

Ex:

`db.student.findOne({name: "Amit"})`

```
College> db.student.findOne({name: "Amit"})
{
  _id: ObjectId('698efda7058d45b47363b112'),
  name: 'Amit',
  age: 23,
  marks: 343
}
```

Querying Data: where (RDBMS)

Syntax: `db.collection_name.find({<key>: <value>})`

Ex:

`db.student.find({age: 23})`

```
College> db.student.find({age:23})
[
  {
    _id: ObjectId('698efda7058d45b47363b112'),
    name: 'Amit',
    age: 23,
    marks: 343
  },
  {
    _id: ObjectId('698efea3058d45b47363b116'),
    name: 'Mohan',
    age: 23,
    marks: 231
  }
]
```

Projection: selecting columns

- ❖ Projection means selecting one or more necessary data rather than selecting whole data.
- ❖ Both find and findOne methods accept a second parameter called projection.
- ❖ This parameter is an object that describes which fields to include in the results.

Syntax: *db.collection_name.find(<query filter>, <projection document>)*

Ex:

db.student.find({}, {name: 1})

Projection: selecting columns

Ex:

```
db.student.find({}, {age: 1})
```

```
College> db.student.find({}, {age: 1})  
[  
  { _id: ObjectId('698efda7058d45b47363b112'), age: 23 },  
  { _id: ObjectId('698efe0a058d45b47363b113'), age: 21 },  
  { _id: ObjectId('698efe0a058d45b47363b114'), age: 22 },  
  { _id: ObjectId('698efe0a058d45b47363b115'), age: 24 },  
  { _id: ObjectId('698efea3058d45b47363b116'), age: 23 }  
]
```

Projection: selecting columns

- ❖ By default `_id` column key will include with every result so to remove `_id` from result use

`_id: 0`

in projection field.

Ex:

```
db.student.find({}, {age: 1, _id: 0})
```

```
College> db.student.find({}, {age: 1, _id: 0})  
[ { age: 23 }, { age: 21 }, { age: 22 }, { age: 24 }, { age: 23 } ]
```

Projection: selecting columns

Ex:

```
db.student.find({}, {age: 1, _id: 0, name: 1})
```

```
College> db.student.find({}, {age: 1, _id: 0, name: 1})  
[  
  { name: 'Amit', age: 23 },  
  { name: 'Amar', age: 21 },  
  { name: 'Raman', age: 22 },  
  { name: 'Suman', age: 24 },  
  { name: 'Mohan', age: 23 }  
]
```

Projection: selecting columns

Ex:

```
db.student.findOne({}, {age: 0})
```

```
College> db.student.findOne({}, {age: 0})  
{ _id: ObjectId('698efda7058d45b47363b112'), name: 'Amit', marks: 343 }
```

Projection: selecting columns

Ex:

```
db.student.find({age: 23}, {marks: 0})
```

```
College> db.student.find({age: 23}, {marks: 0})  
[  
  { _id: ObjectId('698efda7058d45b47363b112'), name: 'Amit', age: 23 },  
  { _id: ObjectId('698efea3058d45b47363b116'), name: 'Mohan', age: 23 }  
]
```

Projection: selecting columns

- ❖ We will get an error if we try to specify both 0 and 1 in the same object.

Ex:

```
db.student.find({}, {age: 1, name: 0})
```

```
College> db.student.find({}, {age: 1, name: 0})  
MongoServerError[Location31254]: Cannot do exclusion  
on field name in inclusion projection
```

Update Document

- ❖ MongoDB's `updateOne()` and `updateMany()` methods are used to update document into a collection.
- ❖ There are 3 methods to update documents into a MongoDB database.
 - ✓ `update()` **// deprecated**
 - ✓ `updateOne()`
 - ✓ `updateMany()`
- ❖ The first parameter is a query object to define which document or documents should be updated.
- ❖ The second parameter is an object defining the updated data.

Syntax: `db.collection_name.update(SELECTION_CRITERIA, UPDATE_DATA, OPTIONS)`

Update Document

db.student.update({name:'Rohan'},{\$set:{name:'Raman'}})

```
College> db.student.findOne({name:"Rohan"})
{
  _id: ObjectId('698efe0a058d45b47363b114'),
  name: 'Rohan',
  age: 22,
  marks: 231
}
College> db.student.update({name:'Rohan'},{$set:{name:'Raman'}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

```
College> db.student.findOne({name:"Raman"})
{
  _id: ObjectId('698efe0a058d45b47363b114'),
  name: 'Raman',
  age: 22,
  marks: 231
}
```

Update Document

```
db.student.update(  
  { _id: ObjectId('698efe0a058d45b47363b114') },  
  { $set: { marks: 245, age: 21 } }  
)
```

```
College> db.student.update(  
... { _id: ObjectId('698efe0a058d45b47363b114') },  
... { $set: { marks: 245, age: 21 } } )  
{  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 1,  
  modifiedCount: 1,  
  upsertedCount: 0  
}  
College> db.student.findOne({ name: "Raman" })  
{  
  _id: ObjectId('698efe0a058d45b47363b114'),  
  name: 'Raman',  
  age: 21,  
  marks: 245  
}
```

Update Document

- ❖ Update multiple document into a collection.

```
db.student.update(  
  {age: 23},  
  {$set: {age: 25}},  
  {multi: true}  
)
```

```
College> db.student.update({ age: 23 },  
  { $set: { age: 25 } },{multi:true} )  
DeprecationWarning: Collection.update()  
  is deprecated. Use updateOne, updateMa  
ny, or bulkWrite.  
{  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 2,  
  modifiedCount: 2,  
  upsertedCount: 0  
}
```

Update Document

updateOne()

- ❖ Updates a single document within the collection based on the filter.
- ❖ This method will update the first document that is found matching the provided query.

Syntax: *db.collection.updateOne(filter, update, options)*

Ex:

db.student.updateOne({age: 23}, {\$set: {age: 25}})

Update Document

Insert if not found

- ❖ If you would like to insert the document if it is not found, you can use the **upsert** option.

Syntax: `db.collection.updateOne(filter, update, {upsert: true} )`

Ex:

```
db.student.updateOne( {age: 29},  
                      {$set: {name: "Ravi", age: 29, marks: 322} },  
                      {upsert: true} )
```

Update Document

updateMany()

- ❖ Updates all documents that match the specified filter for a collection.

Syntax: *db.collection.updateMany(filter, update, options)*

Ex:

db.student.updateMany({age: 23}, {\$set: {age: 25} })

Update Document

updateMany()

- ❖ Update marks on all documents by 20. For this we will use the \$inc (increment) operator:

Ex:

```
db.student.updateMany( {}, { $inc: { marks: 20 } } )
```

Delete Document

- ❖ MongoDB can delete documents by using the methods `deleteOne()` or `deleteMany()`.
- ❖ There are 3 methods to delete documents into a MongoDB database.
 - ✓ `remove()` **// deprecated**
 - ✓ `deleteOne()`
 - ✓ `deleteMany()`
- ❖ **`remove()`** method accepts 2 parameters.
- ❖ One is deletion criteria and second is `justOne` flag.

Syntax: `db.collection_name.remove(<DELETION_CRITERIA>, <justOne>)`

Delete Document

remove()

1. *<DELETION_CRITERIA>*: document deleted according to the deletion criteria.
2. *<justOne>*: if set to true or 1, then remove only 1 document.

Ex:

```
db.student.remove( { _id: ObjectId('698efe0a058d45b47363b114') } )
```

```
db.student.remove( {age: 23}, {justOne: 1} )
```

Delete Document

deleteOne()

- ❖ This method accept a query object. The matching documents will be deleted.
- ❖ The deleteOne() method will delete the first document that matches the query provided.

Ex:

```
db.student.deleteOne( {name: 'Amit'} )
```

Delete Document

deleteMany()

- ❖ This method delete all documents that match the filter from a collection.

Ex:

```
db.student.deleteMany( {age: 25 } )
```

Query Operators

❖ Comparison Query Predicate Operators

Operator	Description
\$eq	Matches values equal to a specified value.
\$gt	Matches values greater than a specified value.
\$gte	Matches values greater than or equal to a specified value.
\$in	Matches any values specified in an array.
\$lt	Matches values less than a specified value.
\$lte	Matches values less than or equal to a specified value.
\$ne	Matches all values not equal to a specified value.
\$nin	Matches if the value is not equal to any of a given list of values.

Query Operators

❖ Comparison Query Predicate Operators

Syntax:

```
db.collection.find( { <field>: { <operator>: <value> } } )
```

Ex:

```
> db.student.find({age:{ $eq:23}})
```

Query Operators

❖ Logical Query Predicate Operators

Name	Description
\$and	Joins query clauses with a logical AND and returns documents that match the conditions of all clauses.
\$nor	Joins query clauses with a logical NOR and returns all documents that fail to match all clauses.
\$not	Inverts the effect of a query predicate and returns documents that do <i>not</i> match the query predicate.
\$or	Joins query clauses with a logical OR and returns all documents that match at least one clause.

Query Operators

❖ Logical Query Predicate Operators

Syntax:

```
db.collection.find( { <logical-operator>: [ { <expression1> }, { <expression2> }, ... ] } )
```

Ex:

```
> db.student.find( { $and: [ {age: 25}, {marks: {$gt:260}}] })
```

Query Operators

- ❖ Between
- ❖ distinct
- ❖ distinct + length
- ❖ null

Query Operators

- ❖ `find().limit()`
- ❖ `find().skip()`
- ❖ `find().sort()`
- ❖ `find().skip().limit()`

Query an Array

Find value from array:

```
> db.collection.find({ field: "value" });
```

Find exact [value] from array:

```
> db.collection.find({ field: ["value" ]});  
> db.collection.find({ field: ["value", "value" ]});
```

Find with \$all: any order / many elements

```
> db.collection.find( { field: { $all : ["value", "value" ] } } );
```

Find with \$op:

```
> db.collection.find( { field: { $op : value } };  
> db.collection.find( { field: { $gt : value, $lt : value } } );
```

Query an Array

Find value by Array Index Position:

```
> db.collection.find({ "field.position": { $op : value }});  
> db.collection.find( { "field.position": { $gt : value, $lt : value }});
```

Query an Array by Array Length:

```
> db.collection.find({ field: { $size : 3 }});
```

Update an Array

\$: The positional \$ operator acts as a placeholder for the first match of the update query document.

Syntax:

```
db.collection.updateOne(  
  { <array>: value ... },  
  { <update operator>: { "<array>.$" : value } }  
)
```

For example:

```
> db.students.updateOne(  
  { _id: 1, grades: 80 },  
  { $set: { "grades.$" : 82 } }  
)
```

Update an Array

`$[]`: The update operator should modify all elements in the specified array field.

Syntax:

```
db.collection.updateOne(  
  { <array>: value ... },  
  { <update operator>: { "<array>.$[]" : value } }  
)
```

For example:

```
> db.students.updateOne(  
  { _id: 1, grades: 80 },  
  { $set: { "grades.$[]" : 82 } }  
)
```

```
db.students.updateMany(  
  { },  
  { $inc: { "grades.$[]": 10 } },  
)
```

Update an Array

\$addToSet: The \$addToSet operator adds a value to an array unless the value is already present, in which case \$addToSet does nothing to that array.

Syntax:

```
db.collection.updateOne(  
  { <array>: value ... },  
  { $addToSet: { <field1>: <value1>, ... } }  
)
```

\$each: Add multiple values to the array field.

```
> db.alphabet.updateOne(  
  { _id: 1 },  
  { $addToSet: { letters: { $each: ["f", "d", "v"] } } }  
)
```

For example:

```
> db.alphabet.updateOne( { _id: 1 }, { $addToSet: { letters: "c" } } )  
> db.alphabet.updateOne( { _id: 1 }, { $addToSet: { letters: ["d", "e"] } } )
```

Update an Array

\$pop: It removes the first or last element of an array. Pass \$pop a value of -1 to remove the first element of an array and 1 to remove the last element in an array.

Syntax:

```
db.collection.updateOne(  
  { <array>: value ... },  
  { $pop: { <field>: <-1 | 1>, ... } }  
)
```

For example:

```
> db.students.updateOne( { _id: 1 }, { $pop: { scores: -1 } } )  
> db.students.updateOne( { _id: 1 }, { $pop: { scores: 1 } } )
```

Update an Array

\$pull: It removes from an existing array all instances of a value or values that match a specified condition.

Syntax:

```
db.collection.updateOne(  
  { <array>: value ... },  
  { $pull: { <field1>: <value|condition>, <field2>: <value|condition>, ... } }  
)
```

For example:

```
> db.students.updateMany( { },  
                           { $pull: { fruits: { $in: [ "apples", "oranges" ] }, vegetables: "carrots" } } )
```

Update an Array

\$push: The \$push operator appends a specified value to an array.

Syntax:

```
db.collection.updateOne(  
  { <array>: value ... },  
  { $push: { <field1>: <value1>, ... } }  
)
```

For example:

```
> db.students.updateOne(  
  { _id: 1 },  
  { $push: { scores: 89 } }  
)
```

Query Operators

❖ \$regex

Syntax:

```
db.collection.find({ field: { $regex: "pattern", $options: "options" } });
```

❖ without \$regex

Syntax:

```
db.collection.find({ field: /pattern/ });
```

Indexes

- ❖ MongoDB indexes are special data structures.
- ❖ **Speed up query and sort operations.**
- ❖ Without them, MongoDB must perform a slow "collection scan," checking every document to find matches.
- ❖ Indexing improves the performance of:
 - **Find queries** (db.collection.find()).
 - **Range queries** (e.g., queries with <, >, <=, >= operators).
 - **Sorting** (e.g., db.collection.find().sort()).
 - **Aggregation operations** involving filtering, grouping, and sorting.

Indexes

Create Index

Ex:

```
> db.products.createIndex({ name: 1})
```

```
> db.products.find({ name: 'Product2000'}).explain('executionStats')
```

Get Indexes

Ex:

```
> db.products.getIndexes()
```

Drop Index

Ex:

```
> db.products.dropIndex('name_1')
```